MAT61-1

Optimization

Albert School — 30 hours

Preface

Almost everything you have studied so far was, secretly, preparation for this course. In **Foundations** (MAT11-1) you learned to differentiate, to read the sign of a second derivative as convexity, and to approximate a function near a point by its Taylor polynomial — the order-1 and order-2 expansions that this whole course runs on. In **Linear Algebra I–III** you learned to write a system of linear relations as $Ax = b$, to read a matrix as a linear map, and finally — in **Euclidean spaces** (MAT31-1) — to diagonalize a symmetric matrix orthogonally and to classify a quadratic form as positive definite, semi-definite, or indefinite from the signs of its eigenvalues. That last skill is not a footnote here; it is the engine of the entire theory. A Hessian is a symmetric matrix, and whether an optimization problem is easy or hard is decided by the signs of its eigenvalues.

Optimization is the discipline of making the best decision subject to constraints. "Best" means we have a single number — a **cost** to minimize or a **profit** to maximize — and "constraints" means we are not free to choose anything we like. Stated this abstractly the problem sounds impossibly general, and indeed in full generality it is hopeless. The art of the subject, and the spine of this book, is learning to recognize the special structure — **linearity**, then **convexity** — that turns a hopeless problem into a solvable one, and learning what to do when that structure is absent.

We will travel in one continuous arc. We begin with **linear programming**, where both objective and constraints are linear and the geometry is so rigid that the optimum must sit at a corner. We then isolate the property that made it work — **convexity** — and study it in its own right, culminating in the KKT conditions, the universal certificate of optimality for constrained problems. With convexity in hand we turn to **algorithms**: gradient descent and the theory that predicts its speed, then its stochastic and adaptive descendants (SGD, Adam) that train modern machine-learning models, then **proximal methods** for objectives that are not even differentiable. We close with the wilderness of **non-convex** optimization, where guarantees evaporate and we must reason about saddle points, high-dimensional landscapes, and the heuristics that nonetheless succeed in practice.

Throughout, one idea recurs like a refrain: **first-order conditions**. In unconstrained calculus you already met it as Fermat's rule, $f'(c) = 0$. Linear programming refines it into complementary slackness; convex theory generalizes it into the KKT conditions; and every algorithm in the second half is a different way of **chasing** a point where the first-order condition holds. Keep that thread in view and the course will feel like one idea told ten ways.

Contents

1 The shape of an optimization problem

1.1 A concrete decision first

A small workshop makes two products, tables and chairs. A table sells for a profit of €30, a chair for €20. Each table needs 4 hours of carpentry and 2 hours of finishing; each chair needs 2 hours of carpentry and 4 hours of finishing. This week there are 100 carpentry hours and 80 finishing hours available. How many tables and chairs should the workshop build to maximize profit?

Before any technique, notice what you instinctively did: you separated three kinds of objects. The **things you get to choose** — the number of tables x_1 and chairs x_2. The **single number you care about** — the profit $30x_1 + 20x_2$. And the **limits you cannot exceed** — carpentry $4x_1 + 2x_2 \le 100$, finishing $2x_1 + 4x_2 \le 80$, and the unspoken $x_1, x_2 \ge 0$ (you cannot build a negative chair). Every optimization problem in this book has exactly this anatomy.

1.2 The general problem and its vocabulary

Definition 1 (Optimization problem) An optimization problem is the task of finding

$$\min_{x \in \mathcal{C}} f(x),$$

where $x \in \mathbb{R}^n$ is the **decision variable** (or **decision vector**), $f : \mathbb{R}^n \to \mathbb{R}$ is the **objective function** (also called the **cost** or **loss**), and $\mathcal{C} \subseteq \mathbb{R}^n$ is the **feasible set** — the points satisfying all constraints. A point $x^* \in \mathcal{C}$ is a **global minimizer** if $f(x^*) \le f(x)$ for every $x \in \mathcal{C}$, and a **local minimizer** if the inequality holds only for x in some neighbourhood of x^*.

Maximizing f is the same as minimizing $-f$, so we lose nothing by always writing problems as minimizations; we will switch freely. The feasible set is usually described by **equality constraints** $h_i(x) = 0$ and **inequality constraints** $g_j(x) \le 0$.

Remark The distinction between local and global minimizers is the whole drama of the course. For a general f an algorithm can get trapped in a local minimum and never discover a deeper one elsewhere. The miracle of convexity, proved in Chapter 3, is that it abolishes this distinction: for a convex problem **every** local minimum is automatically global.

1.3 Why some problems are easy and others impossible

It is worth saying plainly, at the outset, what makes optimization hard. It is **not** the number of variables — we routinely train models with billions of them. It is the **shape** of the objective and the feasible set. When both are convex, the landscape is a single bowl: walk downhill and you cannot help but reach the bottom. When they are not, the landscape can have countless valleys, ridges, and plateaus, and no efficient algorithm is guaranteed to find the lowest point. Convexity, as the tagline promises, is the dividing line. The first half of this book lives on the easy side of that line; the last chapter ventures across it.

2 Linear programming

Linear programming (LP) is optimization in its most rigid and most completely understood form: the objective is linear and every constraint is linear. That rigidity is a gift. It forces the optimum to a corner of the feasible region, gives us a finite algorithm (the simplex method) to

walk between corners, and hands us — for free — a second problem, the **dual**, whose variables turn out to be the economic prices of our resources.

2.1 Formulation

> **Worked example — The workshop as an LP.** The tables-and-chairs problem becomes
>
> $$\begin{aligned} \max_{x_1,x_2} \quad & 30x_1 + 20x_2 \\ \text{subject to} \quad & 4x_1 + 2x_2 \le 100 \\ & 2x_1 + 4x_2 \le 80 \\ & x_1, x_2 \ge 0. \end{aligned}$$
>
> The objective and all four constraints are linear in (x_1, x_2). That is the entire definition of "linear program".

Formulating a real situation as an LP is a skill in its own right, and it always proceeds in the same three steps.

Recipe for LP formulation.

1. **Decision variables.** Name the quantities you control. Decide their units and their sign restrictions.
2. **Objective.** Write the single linear quantity to maximize or minimize as a function of the variables.
3. **Constraints.** For each scarce resource or rule, write a linear inequality or equality. Do not forget implicit constraints such as non-negativity.

Common pitfall The hardest part of formulation is resisting the urge to bake in your guess of the answer. Define variables for **everything you may choose** and let the constraints — not your intuition — carve out what is allowed. A constraint you "obviously" do not need is cheap to include and dangerous to omit.

2.2 Standard form

Different algorithms expect the problem written in a fixed shape. The **standard form** we adopt is

$$\min_x c^T x \quad \text{subject to} \quad Ax = b, \quad x \ge 0,$$

with $b \ge 0$. Two manipulations bring any LP to this form.

- **Inequalities to equalities.** A constraint $a^T x \le \beta$ becomes $a^T x + s = \beta$ with a new **slack variable** $s \ge 0$ measuring the unused amount of that resource. A constraint $a^T x \ge \beta$ becomes $a^T x - s = \beta$ with a **surplus** variable $s \ge 0$.
- **Free variables to non-negative ones.** A variable x with no sign restriction is written $x = x^+ - x^-$ with $x^+, x^- \ge 0$.

> **Worked example — Workshop in standard form.** Maximizing $30x_1 + 20x_2$ is minimizing $-30x_1 - 20x_2$. Adding slacks s_1, s_2 for the two resource constraints gives

$$\begin{aligned} \min \quad & -30x_1 - 20x_2 \\ \text{s.t.} \quad & 4x_1 + 2x_2 + s_1 = 100 \\ & 2x_1 + 4x_2 + s_2 = 80 \\ & x_1, x_2, s_1, s_2 \geq 0. \end{aligned}$$

The slack s_1 is literally "carpentry hours left unused"; at the optimum we will read its value as a clue to which resources are binding.

2.3 The geometry: optima live at corners

Each inequality constraint cuts the plane with a line and keeps one side — a **half-plane**. The feasible set is the intersection of these half-planes: a convex region with flat sides and sharp corners, called a **polyhedron** (a **polytope** when bounded). The objective $c^T x$ is constant along parallel lines (its **level sets**, or **contours**); to minimize it we slide that line as far as possible in the $-c$ direction while still touching the feasible region.

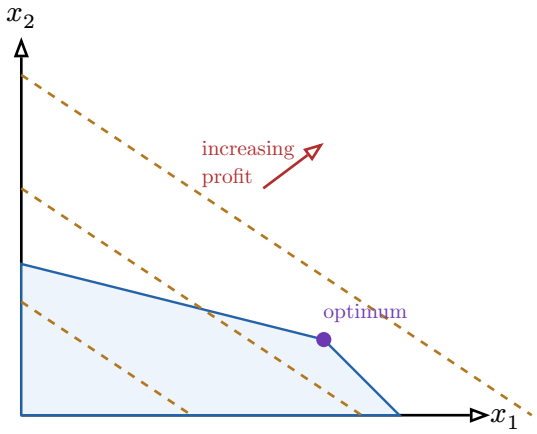


Figure 1: The feasible region of the workshop LP is a polygon. The dashed lines are level sets of the objective $30x_1 + 20x_2$; pushing them in the direction of increasing profit, the last point of contact is a **vertex**. This is no accident: a linear objective over a polyhedron always attains its optimum at a corner (whenever an optimum exists).

Theorem 2 (Fundamental theorem of LP) If a linear program in standard form has an optimal solution, then it has an optimal solution at a **vertex** (an extreme point) of the feasible polyhedron.

The intuition is exactly the picture: a tilted plane over a faceted crystal is highest at a corner, never in the flat interior of a face (unless the whole face ties, in which case a corner of that face still achieves the optimum). This single fact collapses a search over infinitely many feasible points into a search over **finitely many** vertices.

2.4 The simplex method

The simplex method, due to Dantzig, exploits the theorem directly: it walks from vertex to neighbouring vertex, always downhill in cost, until no neighbour is better. Algebraically, a vertex corresponds to a **basic feasible solution** — choose m of the n variables to be **basic** (the rest set to zero), solve the m equality constraints for them, and check the result is non-negative.

Simplex, one iteration.

1. **Start** at a basic feasible solution (a vertex).
2. **Price** the non-basic variables: compute the **reduced cost** of bringing each into the basis — the rate at which the objective would change.
3. **Optimality test.** If every reduced cost is ≥ 0 (for a minimization), no move improves the objective: **stop**, the current vertex is optimal.
4. **Pivot.** Otherwise pick a variable with negative reduced cost to **enter**. Increase it until some basic variable hits zero — that one **leaves**. This is the **ratio test**. You have moved to an adjacent, better vertex. Repeat.

Worked example — One simplex pivot on the workshop. Start at the vertex $x_1 = x_2 = 0$ (build nothing), with slacks $s_1 = 100, s_2 = 80$ basic and cost 0. Both real variables have negative reduced cost (-30 and -20): producing either raises profit. Tables are priced most attractively, so x_1 enters. How far can x_1 grow? Carpentry allows $x_1 \leq \frac{100}{4} = 25$; finishing allows $x_1 \leq \frac{80}{2} = 40$. The binding one is carpentry, so s_1 leaves at $x_1 = 25$. We have moved to the vertex $(25, 0)$ with profit €750. A second pivot brings in x_2 and lands at the optimum $(20, 10)$ with profit €800 — the purple corner in the figure.

Remark In the worst case simplex can visit exponentially many vertices, yet on real-world problems it is famously fast, typically finishing in a number of pivots proportional to the number of constraints. Polynomial-time alternatives (interior-point methods) exist and are preferred for very large problems, but simplex remains the conceptual workhorse and the one that exposes shadow prices most transparently.

2.5 Duality

Every LP, which we now call the **primal**, has a shadow twin called the **dual**. The dual is not a trick; it answers a question you would ask anyway. Return to the workshop and ask: **what is an hour of carpentry worth to me?** Suppose a neighbour offers to rent you their workshop's hours. What is the most you should pay per hour of carpentry, and per hour of finishing, so that renting never makes you worse off than producing yourself?

Assign a price y_1 to a carpentry hour and y_2 to a finishing hour. For the prices to be **consistent** with production, the resources consumed by any product must be worth at least the profit that product earns — otherwise you would rather make the product than sell its resources:

$$4y_1 + 2y_2 \geq 30 \quad \text{(a table's resources)}, \qquad 2y_1 + 4y_2 \geq 20 \quad \text{(a chair's resources)}.$$

Among all consistent price systems you want the **cheapest** total value of your endowment, $100y_1 + 80y_2$. That minimization is the dual:

Definition 3 (Primal–dual pair) For the primal $\max\{c^T x : Ax \leq b, x \geq 0\}$, the dual is

$$\min\{b^T y : A^T y \geq c, y \geq 0\}.$$

Each primal constraint begets a dual variable; each primal variable begets a dual constraint. Maximization becomes minimization, $\leq$ becomes $\geq$, and A is transposed.

Theorem 4 (Weak and strong duality) **Weak duality:** every feasible primal x and feasible dual y satisfy $c^T x \leq b^T y$ — any feasible price system bounds the achievable profit from above. **Strong duality:** if either problem has an optimal solution, so does the other, and their optimal values are **equal**, $c^T x^* = b^T y^*$.

Strong duality is remarkable: the most profit you can squeeze out of your resources equals the least total value any consistent pricing assigns to them. The two views meet exactly.

2.6 Shadow prices and their economic meaning

The optimal dual variables y^* have a name that makes their meaning vivid: **shadow prices**. The shadow price of a constraint is the rate at which the optimal objective would improve if that constraint's right-hand side were relaxed by one unit.

Proposition 5 (Shadow price interpretation) If the resource limit b_i is increased to $b_i + \Delta$, then for small enough Δ the optimal objective changes by exactly $y_i^* \Delta$. In symbols, $y_i^* = \frac{\partial \text{ (optimal value)}}{\partial b_i}$.

Worked example — What is an hour worth?. Solving the workshop dual gives $y_1^* = 7$ and $y_2^* = 1$. So one extra carpentry hour would raise profit by €7, while one extra finishing hour would raise it by only €1. If overtime carpentry costs less than €7/hour, buy it; if it costs more, decline. The shadow price is precisely the break-even price for the resource — which is why the dual variables **are** prices.

Common pitfall A shadow price is a **local** rate, valid only while the same set of constraints stays binding. Add a hundred carpentry hours and eventually finishing becomes the bottleneck; the €7 figure stops applying. Always quote the range over which a shadow price holds.

2.7 Complementary slackness

Weak and strong duality have a sharp logical consequence that links the two problems variable-by-variable. It is the LP incarnation of the first-order condition, and it is the tool you use to **verify** a candidate solution by hand.

Theorem 6 (Complementary slackness) Feasible x^* (primal) and y^* (dual) are **both optimal** if and only if for every index:

- $y_i^* > 0 \Longrightarrow$ the i-th primal constraint is **tight** (holds with equality), and conversely a **slack** primal constraint forces $y_i^* = 0$;
- $x_j^* > 0 \Longrightarrow$ the j-th dual constraint is tight, and a slack dual constraint forces $x_j^* = 0$.

In plain words: **a resource with leftover slack has zero shadow price, and a resource with a positive price must be fully used.** This is common sense made into a theorem — you would not pay for something you are not exhausting.

Worked example — Reading slackness at the workshop optimum. At $x^* = (20, 10)$: carpentry use is $4(20) + 2(10) = 100$ — tight, so its price $y_1^* = 7 > 0$ is allowed. Finishing use is $2(20) + 4(10) = 80$ — also tight, so $y_2^* = 1 > 0$ is allowed. Both products are made $(x_1^*, x_2^* >$

0), so both dual constraints must be tight: $4(7) + 2(1) = 30$ ✓ and $2(7) + 4(1) = 18 \neq 20$. The second fails — so $(7, 1)$ is **not** the exact dual optimum, and the slackness check has caught it. (Resolving the dual exactly gives prices satisfying both tight constraints; the lesson is that slackness is a precise certificate, not a vague heuristic.)

3 Convexity

Linear programming was easy because lines and half-planes are rigid. Convexity is the exact generalization that keeps the essential rigidity — **no false bottoms** — while allowing curvature. This chapter is the theoretical heart of the book. Everything after it is algorithms for convex (or nearly convex) problems.

3.1 Convex sets

Definition 7 (Convex set) A set $\mathcal{C} \subseteq \mathbb{R}^n$ is **convex** if, for any two points $x, y \in \mathcal{C}$, the entire segment between them lies in $\mathcal{C}$:

$$\forall t \in [0, 1], \quad tx + (1 - t)y \in \mathcal{C}.$$

The expression $tx + (1 - t)y$ is a **convex combination**: as t runs from 0 to 1 it traces the straight segment from y to x. So convexity says: pick any two members, and everyone "between" them is a member too.

convex: chord stays inside

non-convex: chord exits

Figure 2: Left: a disk is convex — every chord between two of its points lies inside. Right: an L-shaped region is not convex — the chord between two of its points passes through the notch, leaving the set. Convexity is exactly the "no notches, no holes" property.

Convex sets are closed under intersection: the feasible set of an LP is convex precisely because it is an intersection of half-planes, each of which is convex. This is why "intersect more linear constraints" never breaks convexity.

3.2 Convex functions

You already met convexity of functions in **Foundations** as "the graph lies above its tangent lines" and "$f'' \geq 0$". We now state the definition that works in any dimension, where there is no single second derivative but a whole matrix of them.

Definition 8 (Convex function) A function $f : \mathcal{C} \to \mathbb{R}$ on a convex set $\mathcal{C}$ is **convex** if for all $x, y \in \mathcal{C}$ and $t \in [0, 1]$,

$$f(tx + (1 - t)y) \leq tf(x) + (1 - t)f(y).$$

It is **strictly convex** if the inequality is strict ($<$) whenever $x \neq y$ and $t \in (0, 1)$.

The left side is the function evaluated on the segment; the right side is the straight chord joining $(x, f(x))$ to $(y, f(y))$. So convexity is exactly: **the graph never rises above its own chords.**

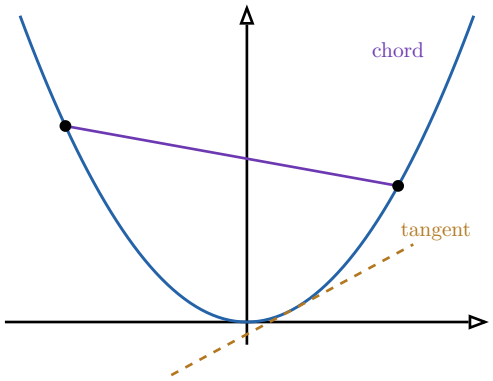


Figure 3: A convex function lies **below** every chord (purple) and **above** every tangent line (dashed). The two phrasings are equivalent and each gives a practical test. The tangent picture is the one that powers gradient methods: a convex function is globally underestimated by its linearization.

3.3 The second-order condition

Checking the chord inequality for all pairs is impractical. For twice- differentiable functions there is a pointwise test, and this is where MAT31-1 pays off. The role of f'' is played by the **Hessian**, the symmetric matrix of second partial derivatives

$$\nabla^2 f(x) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}.$$

Theorem 9 (Second-order condition for convexity) Let f be twice continuously differentiable on an open convex set. Then f is convex $\iff$ its Hessian $\nabla^2 f(x)$ is **positive semi-definite** (PSD) at every x — that is, all its eigenvalues are ≥ 0. If the Hessian is **positive definite** (all eigenvalues > 0) everywhere, then f is strictly convex.

This is the multidimensional sibling of "$f'' \geq 0$". A symmetric matrix is PSD exactly when $v^T(\nabla^2 f)v \geq 0$ for all v, which says the curvature is non-negative in **every** direction — the surface bends up no matter which way you slice it.

Worked example — Verifying convexity via the Hessian. Let $f(x_1, x_2) = x_1^2 + 3x_2^2 - x_1 x_2$. Then

$$\nabla f = \begin{pmatrix} 2x_1 - x_2 \\ 6x_2 - x_1 \end{pmatrix}, \qquad \nabla^2 f = \begin{pmatrix} 2 & -1 \\ -1 & 6 \end{pmatrix}.$$

The Hessian is constant. Its eigenvalues solve $(2-\lambda)(6-\lambda) - 1 = 0$, i.e. $\lambda^2 - 8\lambda + 11 = 0$, giving $\lambda = 4 \pm \sqrt{5}$, both positive. So f is strictly convex on all of $\mathbb{R}^2$. (You could equally have used the quick 2×2 test: symmetric with positive diagonal and positive determinant $2 \cdot 6 - 1 = 11 > 0 \to$ positive definite.)

Common pitfall PSD is **not** "all entries non-negative", and definiteness is **not** "positive determinant". The matrix $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ has positive entries on the diagonal but a negative eigenvalue, so the function $x_1^2 - x_2^2$ it represents is **not** convex — it is a saddle. Always reason about **eigenvalue signs** (or the full chain of leading-principal-minor tests), never about entries.

3.4 Convexity-preserving operations

You rarely check a complicated objective from scratch. Instead you build it from simple convex pieces using operations that **preserve** convexity, exactly as you build complex numbers from 1 and i. Learning these rules turns convexity verification into bookkeeping.

Proposition 10 (A toolbox of convexity-preserving operations) If f and g are convex, then so are:

- **Non-negative weighted sums:** $\alpha f + \beta g$ for $\alpha, \beta \geq 0$.
- **Pointwise maximum:** $\max\{f, g\}$ (and the supremum of any family).
- **Composition with an affine map:** $x \mapsto f(Ax + b)$.
- **Composition with a non-decreasing convex outer function:** $h(f(x))$ when h is convex and non-decreasing.

Worked example — Building a convex objective. Consider the regularized least-squares loss central to data science,

$$f(x) = \|Ax - b\|_2^2 + \lambda \|x\|_1, \qquad \lambda \geq 0.$$

The first term is the convex function $\|\cdot\|_2^2$ composed with the affine map $x \mapsto Ax - b$ — convex. The ℓ_1 norm $\|x\|_1 = \sum_i |x_i|$ is a sum of the convex functions $|x_i|$ — convex. A non-negative weighted sum of two convex functions is convex. Therefore f is convex, with no Hessian computation at all. (We will minimize exactly this f with proximal methods in Chapter 8.)

3.5 Strict convexity and uniqueness of the minimizer

Convexity guarantees that local minima are global; **strict** convexity adds that there is at most one of them. The proof is a clean two-line argument worth internalizing.

Theorem 11 (Uniqueness under strict convexity) Let f be strictly convex on a convex set $\mathcal{C}$. Then f has **at most one** global minimizer on $\mathcal{C}$.

Proof. Suppose two distinct global minimizers $x \neq y$ both attain the minimum value m. Their midpoint $z = \frac{1}{2}x + \frac{1}{2}y$ lies in $\mathcal{C}$ (it is convex), and by **strict** convexity

$$f(z) < \frac{1}{2}f(x) + \frac{1}{2}f(y) = \frac{1}{2}m + \frac{1}{2}m = m.$$

So z is strictly better than the supposed minimum — a contradiction. Therefore no two distinct minimizers can exist. □

Remark (Why convexity is the dividing line) Assemble the pieces. For a convex problem: (i) any local minimum is global — because if a nearby point were higher, the chord

inequality would force a lower point between them, contradicting locality; (ii) the set of minimizers is convex; (iii) under strict convexity it is a single point. So "find a point where the gradient vanishes" — a purely **local**, checkable condition — yields the **global** answer. For non-convex f none of this holds: a vanishing gradient certifies nothing about global optimality. This is precisely why the course splits where it does.

4 Optimality conditions for constrained problems

We now write down the universal first-order condition that a constrained optimum must satisfy. It generalizes Fermat's $\nabla f = 0$ to the case where constraints can **push back**, and it is the single most reused result in the rest of your mathematical education — it reappears in support vector machines, in constrained maximum-likelihood, in economics, anywhere a "best subject to rules" problem is posed.

4.1 From Fermat to Lagrange to KKT

For an **unconstrained** minimum, the gradient must vanish: there is no downhill direction. With an **equality** constraint $h(x) = 0$, you may only move along the constraint surface, and the optimum is where the objective's gradient has no component along that surface — i.e. ∇f is parallel to ∇h. That is the method of **Lagrange multipliers**: $\nabla f = \lambda \nabla h$. The KKT conditions extend this to **inequalities**, where a constraint either presses on the solution (active) or sits idle (inactive).

Definition 12 (KKT conditions) For the problem $\min f(x)$ subject to $g_j(x) \le 0$ and $h_i(x) = 0$, form the **Lagrangian**

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \sum_j \mu_j g_j(x) + \sum_i \lambda_i h_i(x).$$

A point x^* with multipliers (μ^*, λ^*) satisfies the **KKT conditions** if:

1. **Stationarity:** $\nabla f(x^*) + \sum_j \mu_j^* \nabla g_j(x^*) + \sum_i \lambda_i^* \nabla h_i(x^*) = 0$.
2. **Primal feasibility:** $g_j(x^*) \le 0$ and $h_i(x^*) = 0$ for all i, j.
3. **Dual feasibility:** $\mu_j^* \ge 0$ for all j.
4. **Complementary slackness:** $\mu_j^* g_j(x^*) = 0$ for all j.

The four conditions are exactly the LP duality story generalized. Stationarity is "no downhill direction once constraints are accounted for". Dual feasibility $\mu_j \ge 0$ says an inequality can only **push**, never pull. Complementary slackness says an **inactive** constraint ($g_j < 0$) carries multiplier zero — the same logic as a slack resource having zero shadow price. The multipliers μ_j are nothing other than shadow prices of the constraints.

Theorem 13 (KKT as necessary and sufficient) Under mild regularity (a **constraint qualification**), the KKT conditions are **necessary** for x^* to be a local minimum of any smooth problem. If in addition the problem is **convex** (convex f and g_j, affine h_i), the KKT conditions are also **sufficient**: any KKT point is a **global** minimizer.

That second sentence is the prize. For a convex problem, solving the KKT system — a set of equations and inequalities — **certifies** a global optimum. This is the abstract engine underneath the simplex method, and the framework machine learning reuses when it constrains models.

Worked example — KKT in one inequality. Minimize $f(x) = (x-3)^2$ subject to $g(x) = x - 1 \le 0$. Stationarity: $2(x-3) + \mu = 0$. Complementary slackness: $\mu(x-1) = 0$, so either $\mu = 0$ or $x = 1$. If $\mu = 0$ then $x = 3$, which violates $x \le 1$ — infeasible. So the constraint is active: $x^* = 1$, giving $\mu^* = 2(3-1) = 4 \ge 0$, dual-feasible. The constraint **binds**, and its shadow price $\mu^* = 4$ is the rate at which the optimal cost would fall if we relaxed the bound from 1 upward toward the unconstrained optimum 3.

5 Convergence theory: the language of speed

We leave exact, finite algorithms behind. From here on we **iterate**: produce a sequence $x_0, x_1, x_2, \ldots$ that we hope converges to a minimizer. The central questions become **does it converge?** and **how fast?** To answer them precisely we need two numbers that summarize the geometry of f: an upper bound on how fast its gradient can change, and a lower bound on its curvature.

5.1 Lipschitz-smooth gradients: the constant L

Definition 14 (L-smoothness) A differentiable f has an ***L*-Lipschitz gradient** (we say f is ***L*-smooth**) if

$$\|\nabla f(x) - \nabla f(y)\| \le L\|x - y\| \quad \text{for all } x, y.$$

Equivalently, when f is twice differentiable, every eigenvalue of the Hessian is at most L: $\nabla^2 f(x) \preceq LI$.

L caps how quickly the slope can swing, so the gradient does not lie about the function for too long. The key consequence is the **descent lemma**: an L-smooth function is bounded above by an upward parabola of curvature L tangent at any point,

$$f(y) \le f(x) + \nabla f(x)^T (y - x) + \frac{L}{2}\|y - x\|^2.$$

This quadratic upper bound is what lets us guarantee that a gradient step makes progress, and it is where the magic step size $\frac{1}{L}$ comes from.

5.2 Strong convexity: the constant μ

Definition 15 (Strong convexity) f is ***μ*-strongly convex** ($\mu > 0$) if $f(x) - \frac{\mu}{2}\|x\|^2$ is convex — equivalently, when twice differentiable, every Hessian eigenvalue is at least μ: $\nabla^2 f(x) \succeq \mu I$. Geometrically, f is bounded **below** by an upward parabola of curvature μ tangent at any point:

$$f(y) \ge f(x) + \nabla f(x)^T (y - x) + \frac{\mu}{2}\|y - x\|^2.$$

Where L bounds curvature from above, μ bounds it from below — the bowl is at least this steep everywhere, so it cannot flatten into a long indecisive trough. Strong convexity forces a **unique** minimizer and, crucially, makes the gradient grow as you move away from it, so a large gradient guarantees you are still far from the optimum.

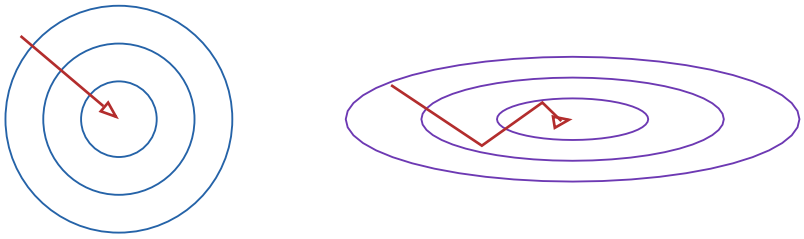
5.3 The condition number κ

Together μ and L trap the curvature in every direction inside the band $\mu \leq$ (curvature) $\leq L$. Their ratio measures how distorted the bowl is.

Definition 16 (Condition number) For an L-smooth, μ-strongly convex function the **condition number** is

$$\kappa = \frac{L}{\mu} \geq 1.$$

When $\kappa = 1$ the level sets are perfect circles (or spheres): the steepest- descent direction points straight at the minimum. When κ is large the level sets are long thin ellipses: the gradient points mostly **across** the valley rather than along it, and naïve descent zig-zags.



$\kappa \approx 1$: straight to the bottom $\kappa \gg 1$: gradient zig-zags

Figure 4: The condition number is the eccentricity of the level sets. Round contours (left) let gradient descent walk straight in; elongated contours (right) make the negative gradient point across the valley, forcing a slow zig-zag. Convergence speed is governed by κ, not by the raw size of L or μ.

5.4 Convergence rates

A **rate** describes how the error $f(x_k) - f(x^*)$ (or $\|x_k - x^*\|$) shrinks with the iteration count k. Two regimes recur throughout the book.

Two vocabularies of speed.

- **Sublinear:** the error decays like a power of $\frac{1}{k}$, e.g. $O(\frac{1}{k})$ or $O(\frac{1}{k^2})$. Each extra digit of accuracy costs more iterations than the last. Typical of merely convex (not strongly convex) problems.
- **Linear (geometric):** the error shrinks by a constant factor each step, $f(x_k) - f(x^*) \leq \rho^k \cdot$ (const) with $\rho < 1$. Each iteration buys a fixed number of digits. Typical of **strongly** convex problems, where $\rho \approx 1 - \frac{1}{\kappa}$.

Read $\rho \approx 1 - \frac{1}{\kappa}$ closely: a large condition number pushes ρ toward 1, so progress per step is tiny. This is the quantitative form of the zig-zag picture and the reason practitioners obsess over **conditioning** (rescaling variables, preconditioning) — it directly buys speed.

6 Gradient descent

We can finally state the workhorse algorithm. Gradient descent embodies the most basic intuition in optimization: to go downhill, step in the direction of steepest descent, which is the **negative gradient**.

6.1 The algorithm

Definition 17 (Gradient descent) Choose a starting point x_0 and step sizes $\eta_k > 0$. Iterate

$$x_{k+1} = x_k - \eta_k \nabla f(x_k).$$

The step size η_k (also called the **learning rate**) controls how far to move along the negative gradient.

The entire behaviour of the method hinges on η_k. Too small and progress is glacial; too large and the steps overshoot the valley and the iterates diverge. The smoothness constant L tells us exactly where the boundary is.

6.2 Choosing the step size

Proposition 18 (The 1/L rule) If f is L-smooth, the constant step size $\eta = \frac{1}{L}$ guarantees descent at every step:

$$f(x_{k+1}) \le f(x_k) - \frac{1}{2L}\|\nabla f(x_k)\|^2.$$

Each iteration strictly decreases f by an amount proportional to the squared gradient norm.

Proof (sketch). Insert the step $y = x_k - \frac{1}{L}\nabla f(x_k)$ into the descent-lemma upper bound $f(y) \le f(x_k) + \nabla f(x_k)^T(y - x_k) + \frac{L}{2}\|y - x_k\|^2$. The linear and quadratic terms combine to $-\frac{1}{L}\|\nabla f(x_k)\|^2 + \frac{1}{2L}\|\nabla f(x_k)\|^2 = -\frac{1}{2L}\|\nabla f(x_k)\|^2$, which is the claim. □

The $\frac{1}{L}$ rule is beautiful but requires knowing L. When L is unknown or varies, we instead **search** for a good step at each iteration.

Definition 19 (Backtracking line search) Fix parameters $\alpha \in \left(0, \frac{1}{2}\right)$ and $\beta \in (0, 1)$. Start with a trial step $\eta = 1$. While

$$f(x_k - \eta \nabla f(x_k)) > f(x_k) - \alpha\eta\|\nabla f(x_k)\|^2,$$

shrink $\eta \leftarrow \beta\eta$. Then take the step with the accepted η.

Line search adapts automatically to the local curvature, demanding only that each step achieve a fraction α of the decrease the gradient predicts. It is the practical default when L is not handed to you.

6.3 Convergence analysis

Summing the per-step decreases from the $\frac{1}{L}$ rule yields the global guarantees, which are exactly the two rate regimes of the previous chapter.

Theorem 20 (Gradient descent convergence) With step size $\frac{1}{L}$:

- If f is convex and L-smooth, then $f(x_k) - f(x^*) \le \frac{L\|x_0 - x^*\|^2}{2k}$ — a **sublinear** $O\left(\frac{1}{k}\right)$ rate.
- If in addition f is μ-strongly convex, then

$$\|x_k - x^*\|^2 \leq \left(1 - \frac{1}{\kappa}\right)^k \|x_0 - x^*\|^2$$

— a **linear** rate with factor $1 - \frac{1}{\kappa}$.

The strongly convex case is the headline: convergence in roughly $\kappa \log\left(\frac{1}{\varepsilon}\right)$ iterations to reach error ε. Halve the condition number and you halve the work. This is the precise statement of the strongly convex guarantee, and the reason every later algorithm is, at heart, an attempt to beat the factor $1 - \frac{1}{\kappa}$.

Common pitfall Do not set the learning rate "as large as seems to still work" and forget it. If $\eta > \frac{2}{L}$ the simplest quadratic already **diverges** — the iterates blow up geometrically. When training diverges to `NaN`, an over-large learning rate is the first suspect.

7 Stochastic gradient descent

Gradient descent needs the full gradient $\nabla f(x_k)$. In data science the objective is almost always an **average over data**,

$$f(x) = \frac{1}{N} \sum_{i=1}^{N} f_i(x),$$

where f_i is the loss on the i-th of N training examples and N can be in the millions or billions. Computing ∇f once means touching **every** example — far too expensive to do per step. Stochastic gradient descent (SGD) replaces the exact gradient with a cheap random estimate.

7.1 The unbiased estimator

Definition 21 (Stochastic gradient) At each step, draw an index i uniformly at random from $\{1, \ldots, N\}$ and use $\nabla f_i(x_k)$ in place of $\nabla f(x_k)$:

$$x_{k+1} = x_k - \eta_k \nabla f_i(x_k).$$

The single most important fact justifying this is that the random gradient is **right on average**.

Proposition 22 (Unbiasedness) If i is drawn uniformly, then

$$\mathbb{E}_i[\nabla f_i(x)] = \frac{1}{N} \sum_{i=1}^{N} \nabla f_i(x) = \nabla f(x).$$

The stochastic gradient is an **unbiased estimator** of the true gradient.

Proof. The expectation of a function of a uniform index is the plain average of its values: $\mathbb{E}_i[\nabla f_i(x)] = \sum_{i=1}^{N} \frac{1}{N} \nabla f_i(x)$, which is exactly $\nabla f(x)$ by definition of f. Linearity of expectation does the rest. □

So each SGD step moves in the right direction **on average**, even though any single step is noisy. Unbiasedness is what lets the noise average out over many steps instead of accumulating into a systematic error.

7.2 Mini-batches and the variance trade-off

A single example gives a very noisy estimate. The standard compromise is a **mini-batch**: average the gradient over a small random subset $\mathcal{B}$ of examples,

$$g_{\mathcal{B}}(x) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla f_i(x).$$

It is still unbiased, but its **variance** is smaller by roughly a factor $\frac{1}{|\mathcal{B}|}$. Batch size thus tunes a trade-off: larger batches give less noise per step (steadier progress) but cost more computation per step.

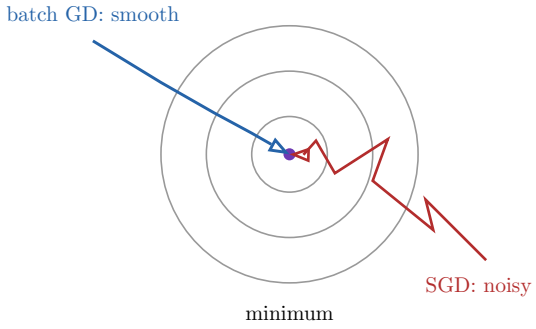


Figure 5: Batch gradient descent (blue) follows a smooth path because each step uses the exact gradient. SGD (red) jitters because each step uses a noisy unbiased estimate, yet it still drifts toward the minimum — and each of its steps is $\frac{N}{|\mathcal{B}|}$ times cheaper. On large datasets the noisy-but-cheap trade is overwhelmingly worth it.

7.3 Variance reduction and comparison with batch GD

Because of the persistent noise, plain SGD with a constant step does not settle at the minimum; it hovers in a noise ball around it. Two remedies:

- **Decreasing step sizes** $\eta_k \to 0$ (for example $\eta_k \propto \frac{1}{k}$) shrink the noise ball to a point, recovering convergence — at the price of a slower $O\left(\frac{1}{k}\right)$ rate.
- **Variance-reduction methods** (SVRG, SAG, SAGA) occasionally compute a full gradient and use it as a **control variate** to cancel most of the noise. They restore the fast **linear** rate of batch GD while keeping the per-step cost of SGD — the best of both worlds for finite-sum problems.

Batch GD vs. SGD. Batch GD: exact gradient, smooth descent, linear rate, but $O(N)$ cost per step. SGD: noisy gradient, cheap $O(|\mathcal{B}|)$ steps, but a noise floor without decreasing steps or variance reduction. On massive datasets, SGD reaches a good solution in far less wall-clock time, which is why it — not batch GD — trains modern models.

8 The Adam optimizer

SGD treats every coordinate identically with one global step size. But in a deep model some parameters see large, frequent gradients and others tiny, rare ones — they deserve different step sizes. **Adam** (Adaptive Moment Estimation) is the optimizer that, by default, trains most of today's neural networks. It maintains running averages of the gradient and of its square, and uses them to give every parameter its own adaptive step.

8.1 First and second moment estimates

Definition 23 (Adam's moment estimates) Let $g_k = \nabla f_i(x_k)$ be the stochastic gradient at step k. With decay rates $\beta_1, \beta_2 \in [0, 1)$ (defaults 0.9 and 0.999), Adam keeps two exponential moving averages, updated elementwise:

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k, \qquad v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2.$$

Here m_k estimates the **first moment** (the mean gradient) and v_k the **second moment** (the mean squared gradient), with g_k^2 taken elementwise.

The first moment m_k is **momentum**: it averages recent gradients, smoothing the noise and carrying velocity through small bumps. The second moment v_k gauges how large each coordinate's gradient typically is.

8.2 Bias correction

Both averages are initialized at zero, so early on they are biased toward zero — m_k and v_k start too small simply because the running average has not yet "filled up". Adam corrects this exactly.

Definition 24 (Bias-corrected estimates)

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k}, \qquad \hat{v}_k = \frac{v_k}{1 - \beta_2^k}.$$

The factor $1 - \beta_1^k$ is precisely the total weight accumulated in the moving average after k steps; dividing by it rescales the estimate to be unbiased. As k grows, $\beta_1^k \to 0$ and the correction fades to 1, so it matters only in the early steps — exactly when it is needed.

8.3 Per-parameter adaptation

Definition 25 (The Adam update) With learning rate η and a small ε (e.g. 10^{-8}) for numerical safety, every parameter is updated by

$$x_{k+1} = x_k - \eta \cdot \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \varepsilon},$$

with all operations elementwise.

This is the heart of Adam. Dividing the smoothed gradient $\hat{m}_k$ by $\sqrt{\hat{v}_k}$ **normalizes each coordinate by its own typical magnitude**: a parameter with consistently large gradients gets a damped effective step, while a parameter with tiny gradients gets an amplified one. The net effect is a per-parameter learning rate that adapts automatically — which is why Adam works well across wildly different layers without painstaking manual tuning.

Remark The combination of momentum ($\hat{m}$) and per-coordinate scaling ($\frac{1}{\sqrt{\hat{v}}}$) makes Adam behave gracefully on the ill-conditioned, noisy, non-convex landscapes of deep learning. It is not a magic bullet — for some problems well-tuned SGD with momentum generalizes better — but as a robust default that "just works", Adam is unmatched.

9 Proximal methods

Every algorithm so far assumed a **differentiable** objective. But the most useful regularizer in data science, the ℓ_1 norm $\|x\|_1 = \sum_i |x_i|$, has a corner at zero where no derivative exists — and that corner is exactly the feature we want, because it drives coordinates to **exactly** zero, producing sparse models. Proximal methods are gradient descent's extension to objectives that are convex but only partly smooth.

9.1 The proximal operator

The idea: split the objective into a smooth part f and a simple but possibly non-smooth part g, $\min_x f(x) + g(x)$. Handle f with a gradient step and g with a **proximal step**.

Definition 26 (Proximal operator) For a convex function g and parameter $\eta > 0$, the **proximal operator** is

$$\operatorname{prox}_{\eta g}(v) = \arg\min_x \left(g(x) + \frac{1}{2\eta}\|x - v\|^2 \right).$$

It returns the point that balances making g small against staying close to v.

The proximal operator is a "generalized gradient step": for a smooth g it nearly reproduces a gradient step, but it remains well-defined even when g has corners. It can be read as an implicit step that asks "reduce g, but do not stray too far from where I am".

9.2 Soft thresholding: the prox of the ℓ_1 norm

The single most important proximal operator is the one for $g(x) = \lambda\|x\|_1$, because it has a closed form and it is what makes ℓ_1-regularized problems solvable.

Proposition 27 (Soft thresholding) The proximal operator of $g(x) = \lambda\|x\|_1$ acts coordinatewise as the **soft-thresholding** function

$$\operatorname{prox}_{\eta\lambda\|\cdot\|_1}(v)_i = \operatorname{soft}_{\eta\lambda}(v_i) = \begin{cases} v_i - \eta\lambda & \text{if } v_i > \eta\lambda \\ 0 & \text{if } |v_i| \le \eta\lambda \\ v_i + \eta\lambda & \text{if } v_i < -\eta\lambda \end{cases}.$$

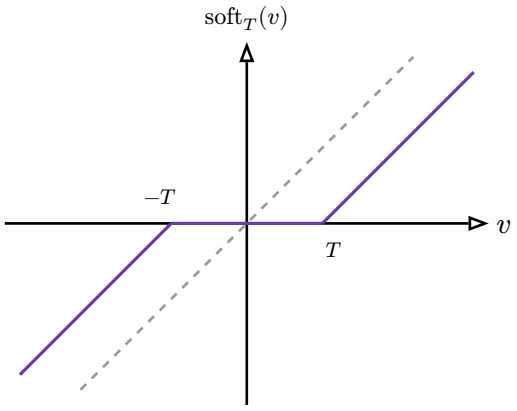


Figure 6: Soft thresholding with threshold $T = \eta\lambda$. Inputs in $[-T, T]$ are snapped to **exactly zero** (the flat segment); larger inputs are kept but shrunk toward zero by T. The dead zone around the origin is what makes ℓ_1 regularization produce sparse solutions — small coefficients are eliminated outright rather than merely shrunk.

Remark Contrast soft thresholding with the **shrinkage** of ℓ_2 regularization, which multiplies every coordinate by a factor < 1 but never sets any to zero. The flat dead-zone of soft thresholding is precisely why ℓ_1 yields **sparse** models and ℓ_2 does not — a distinction at the heart of Lasso regression.

9.3 ISTA and FISTA

Combining a gradient step on the smooth part with a proximal step on the non-smooth part gives the **Iterative Shrinkage-Thresholding Algorithm** (ISTA):

$$x_{k+1} = \operatorname{prox}_{\eta g}(x_k - \eta \nabla f(x_k)).$$

Take a normal gradient step on f, then soft-threshold. ISTA converges at the sublinear $O\left(\frac{1}{k}\right)$ rate — the same as plain gradient descent on a non-smooth convex problem.

FISTA (the "Fast" version) adds a momentum term, extrapolating each iterate slightly past the previous one before applying the prox step. With this single modification — due to Nesterov's acceleration — the rate improves to $O\left(\frac{1}{k^2}\right)$, a dramatic speed-up for the same per-iteration cost.

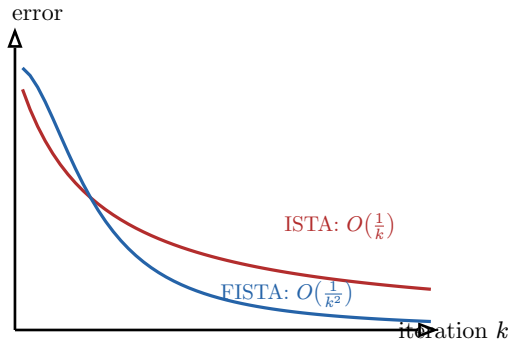


Figure 7: Error versus iteration for ISTA ($O\left(\frac{1}{k}\right)$) and FISTA ($O\left(\frac{1}{k^2}\right)$). Adding one momentum term turns the slow upper curve into the fast lower one at no extra per-step cost — to reach error ε, FISTA needs about $\frac{1}{\sqrt{\varepsilon}}$ iterations where ISTA needs $\frac{1}{\varepsilon}$.

9.4 ADMM for structured problems

Some problems split naturally into pieces coupled by a linear constraint, e.g. $\min_{x,z} f(x) + g(z)$ subject to $Ax + Bz = c$. The **Alternating Direction Method of Multipliers** (ADMM) alternates between minimizing over x, minimizing over z, and a dual update of the multiplier for the coupling constraint — each subproblem often having a clean proximal solution. ADMM shines when a problem decomposes into parts that are individually easy but awkward together, and it parallelizes naturally across those parts.

10 Non-convex optimization

We finally cross the dividing line. When f is non-convex — as every deep neural network loss is — the clean guarantees vanish. A vanishing gradient no longer certifies a global minimum; it certifies only a **critical point**, which might be a local minimum, a local maximum, or something subtler and, in high dimensions, far more common: a saddle point. This chapter is a field guide to that wilderness.

10.1 Local minima and saddle points

At a critical point ($\nabla f = 0$) the **Hessian's eigenvalue signs** classify the geometry — the same MAT31-1 classification that defined convexity, now used locally:

- all eigenvalues > 0 (Hessian positive definite): **local minimum** — a bowl;
- all eigenvalues < 0 (negative definite): **local maximum** — a dome;
- **mixed** signs (indefinite): a **saddle point** — up in some directions, down in others.

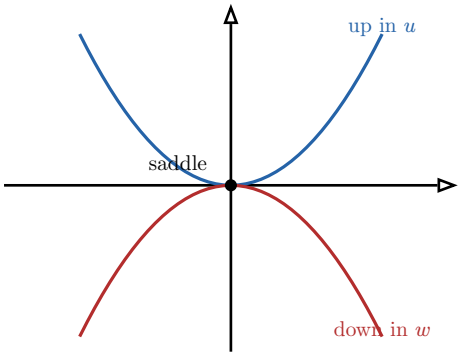


Figure 8: A saddle point seen along its two principal directions. The gradient vanishes here, yet it is no minimum: the function curves **up** along direction u (positive Hessian eigenvalue) and **down** along direction w (negative eigenvalue). The mixed signs are the signature of a saddle.

10.2 Why saddles dominate in high dimensions

A subtle and important fact: in high-dimensional problems, critical points are **overwhelmingly** saddle points rather than local minima. The heuristic is a counting argument. At a random critical point in n dimensions, each of the n Hessian eigenvalues is positive or negative roughly independently. For a local **minimum** you need **all** n to be positive — an event that becomes astronomically unlikely as n grows (think 2^{-n}). A saddle needs only **one** sign disagreement, which is almost certain. So as dimension grows, minima become exceedingly rare and saddles become the norm.

> **Remark** This reframed the entire theory of why deep learning works. Early fears that training would stall in bad local minima were largely misplaced: the real obstacle is **saddle points**, where the gradient is small and progress crawls. And the good news is that saddles, unlike local minima, always offer an escape direction — the one with a negative eigenvalue.

10.3 Subgradient methods

For non-smooth non-convex (or even non-smooth convex) functions where no gradient exists at a point, we generalize the gradient to a **subgradient**: any vector s with $f(y) \geq f(x) + s^T(y - x)$ for all y (for convex f), i.e. the slope of a supporting line. The **subgradient method** iterates $x_{k+1} = x_k - \eta_k s_k$ with s_k any subgradient at x_k. It is robust and simple but slow — only $O\left(\frac{1}{\sqrt{k}}\right)$ even in the convex case — and it requires a carefully decaying step size, since at a corner the subgradient need not vanish at the optimum.

10.4 Strategies for non-convex landscapes

No algorithm can promise the global minimum of a general non-convex function, but a toolbox of pragmatic strategies works well in practice:

Coping with non-convexity.

- **Random restarts.** Run the optimizer from many random initial points and keep the best result. Different basins are explored; the chance of landing in a good one rises with the number of restarts.

- **Noise as escape.** The inherent noise of SGD helps the iterates **jump out** of shallow minima and slide off saddle points — non-convex optimization is one place where SGD's noise is a feature, not a bug.
- **Momentum.** Carrying velocity (as in Adam) powers through flat saddle regions where the gradient is nearly zero.
- **Good initialization.** Careful starting points (e.g. structured weight initialization in neural networks) keep the optimizer in well-behaved regions of the landscape from the outset.

Common pitfall Resist reading a converged non-convex result as "the" optimum. It is one local solution among possibly many. Report it as such, compare several restarts, and be suspicious of any claim of global optimality without convexity or an exhaustive search to back it.

11 Epilogue: one idea, ten voices

Look back over the arc. Linear programming taught us that optimality is certified by a **dual** price system and **complementary slackness**. Convexity revealed **why** such certificates work — local information determines the global answer — and the KKT conditions packaged the certificate for any convex constrained problem. The algorithmic half was a single move, the gradient step, dressed for successively harder weather: scaled by $\frac{1}{L}$ for guaranteed smooth descent, made stochastic for massive data, made adaptive in Adam, made proximal for sparsity, and finally sent, without guarantees but with good heuristics, into the non-convex wild.

The constant companion was the **first-order condition**. Fermat's $\nabla f = 0$, LP's complementary slackness, the KKT stationarity equation, and the fixed point that every iterative method chases are all the same statement: **at the optimum, there is no improving direction left.** And the constant judge of difficulty was the **eigenvalue sign** — of the Hessian for convexity and curvature, of the condition number for speed, of the saddle for the shape of the high-dimensional landscape. Carry those two threads forward; you will meet them again in every machine-learning model you ever train.